TraceFix: A Graph-Native Bug-Resolution Agent with Verified Context and Self-Compiling Memory

Proposed project report for Graph Hacks: Context for AI Agents (WeMakeDevs x FalkorDB, Oct 15-18, 2026). Draft v1. Working name: TraceFix.

1. Executive Summary

Enterprise bugs reach L1/L2 support as tickets, and the real cost is not the fix but finding the right code, the right owner, and the earlier incident that already explained it. Today that knowledge lives in people's heads and scattered tools.

TraceFix puts that knowledge in a FalkorDB graph and builds an agent whose behavior is driven by graph traversals. The agent loads only the small subgraph a ticket needs, proposes a patch, runs tests, and hands the result to a human for approval. Three ideas make it distinct:

1. **Context sufficiency certificates.** After a verified fix, the system ablates its own context and stores the minimal sufficient subgraph that the fix actually needed. This is the auditable path behind every answer.
2. **Memory that compiles into playbooks.** Resolved incidents are stored as path patterns (symptom, code region, cause type, fix shape). Recurring patterns are promoted to playbook nodes that the next ticket checks first.
3. **A learning curve as the headline result.** A chronological replay of real bugs measures tokens loaded, localization accuracy and fix rate as the graph accumulates, against baselines without the graph or without memory.

The design covers all three tracks: Track 01 is the primary entry, Track 02 (memory and coordination) is the learning and multi-agent layer, and Track 03 (company brain) is the ownership, decision and source-tracing layer.

Claims in this report that cannot yet be verified are marked as assumptions in section 17.

2. Problem and Target Use Case

Who: L1/L2 support engineers and the on-call or L3 developers they escalate to.

Pain: a ticket arrives with a symptom and perhaps a stack trace. Someone then spends hours finding the responsible code, its owner, what else it touches, and whether it happened before. Whole-repository context is too large for an agent, and open-ended exploration drifts and wastes tokens.

Target workflow:

1. A ticket is logged with symptom text, logs or a stack trace.
2. The agent localizes the cause by walking a graph, not by reading the repository.
3. It proposes a patch with an evidence path, runs the tests, and opens a pull request.
4. A human approves. The outcome is written back to the graph.
5. Similar future tickets resolve faster because the pattern is remembered.

Non-goals: fully autonomous production changes, general benchmark leaderboard claims, and multi-language support beyond one primary language.

3. Hackathon Requirement Mapping

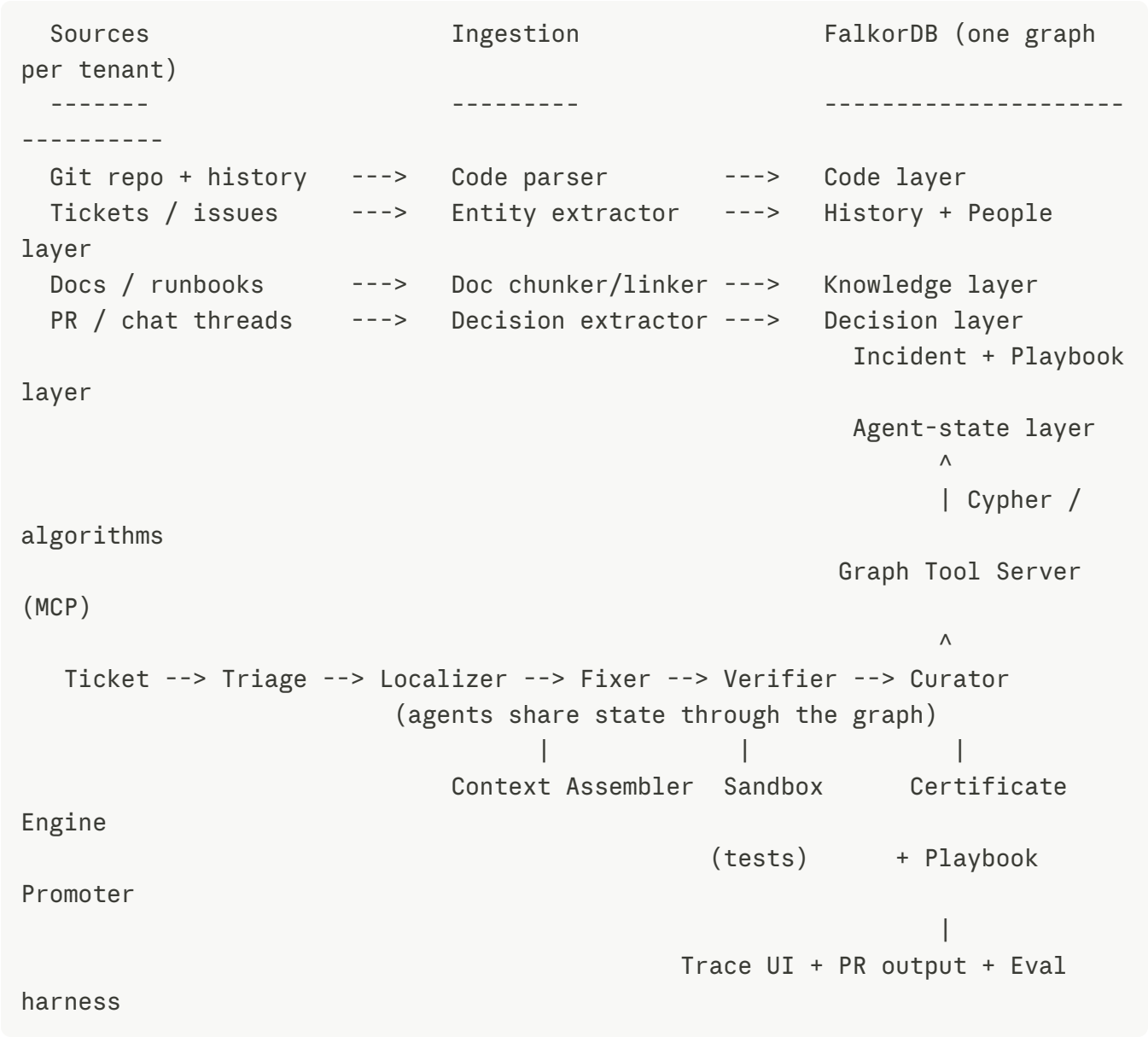
The challenge brief states that the graph has to shape what the agent does, not sit on the side. The table maps each stated track focus to a concrete component.

Track	Stated focus	TraceFix component
01 Agents That Act on Connected Data	Multi-hop reasoning	Bounded multi-hop traversal from ticket to function, test, commit, owner
01	Graph queries and algorithms as agent tools	Deterministic tool layer wrapping Cypher and graph algorithms (shortest path, PageRank/centrality, connected components)
01	Tool selection through MCP	Graph tools exposed through an MCP server; the agent chooses among them per step
01	Actions based on what the graph shows	Patch scope, test selection, reviewer assignment and risk flag all derived from graph results

Track	Stated focus	TraceFix component
01	Explainable results	Evidence path plus the minimal sufficient subgraph certificate
02 Agent Memory and Coordination	Episodic, semantic, procedural memory	Incident nodes (episodic), component and ownership facts (semantic), playbook nodes (procedural)
02	Persistent conversation history	Session and Message nodes linked to tickets
02	Shared state and handoffs	Triage, Localizer, Fixer, Verifier and Curator agents hand off through a Task node and shared subgraph
02	High-concurrency reads and writes	Parallel ticket workers; load test reported in the evaluation
02	Separate memory per user or tenant	One FalkorDB graph per tenant or team
03 Company Brain	Knowledge from docs, tickets, chats	Ingestion of tickets, runbooks, PR discussions and chat exports into one graph
03	Who owns what, who knows what	OWNED_BY and KNOWS edges from git history and resolved incidents
03	Tracing decisions back to source	Decision nodes with SOURCE edges to the originating PR, ticket or document
03	Keeping knowledge current	Content-hash invalidation and valid_from/valid_to on edges
03	Answers with clear sources	Every answer cites node IDs and their sources

The FalkorDB capabilities referenced here (per-tenant graphs in one instance, Cypher, graph algorithms, MCP, Python client, Docker) come from the hackathon page and should be confirmed against FalkorDB documentation before the build starts.

4. System Architecture



Components

- **Ingestion pipeline.** Parses code into Function, File and Class nodes and CALLS, DEFINED_IN, IMPORTS and COVERED_BY edges. Reads git history for commits, authors and file changes. Extracts entities from tickets and documents.
- **FalkorDB layer.** Holds all state. A tenant is one graph, so isolation does not require separate deployments.
- **Graph Tool Server (MCP).** The only way agents touch the graph. Each tool is a pre-verified Cypher query or algorithm, so agents never write raw Cypher in the main loop.

- **Agent runtime.** Five role agents with a thin orchestrator. Handoffs are Task nodes in the graph, not hidden in a prompt.
- **Context Assembler.** Builds the minimal prompt from traversal results under a token budget.
- **Sandbox.** Runs candidate patches and tests in isolation.
- **Certificate Engine.** Ablates context after a verified fix to compute the minimal sufficient subgraph.
- **Playbook Promoter.** Clusters incident path patterns and promotes recurring ones.
- **Trace UI.** Shows the evidence path, loaded snippets, certificate, and the learning curve.
- **Evaluation harness.** Replays bugs chronologically against baselines and writes the metrics.

5. Graph Data Model

Layers and nodes

Layer	Nodes	Purpose
Code	Service, File, Class, Function, Test	Structure and call relationships
History	Commit, Person, Team	Ownership and change recency
Knowledge	Doc, Runbook, Concept	Documentation tied to code
Decision	Decision, Discussion	Why things are the way they are, with sources
Incident (episodic)	Ticket, Incident, Fix	What happened and how it was resolved
Procedural	Playbook, Pattern	Reusable fix strategies compiled from incidents
Agent state	Task, Session, Message, Certificate	Handoffs, history and proofs

Key relationships

Edge	From to	Notes
CALLS, IMPORTS, DEFINED_IN	Code to Code	Static structure
COVERED_BY	Function to Test	Which tests exercise which code
MODIFIED_IN	File/Function to Commit	With timestamps
AUTHORED_BY, OWNED_BY	Commit/File to Person/Team	Ownership from history and CODEOWNERS if present
KNOWS	Person to Concept/Function	Weighted by past fixes and commits
AFFECTS	Ticket to Service/Function	From localization
RESOLVED_BY	Incident to Fix	Verified outcome
INSTANCE_OF	Incident to Pattern	Pattern membership
PROMOTED_TO	Pattern to Playbook	After recurrence threshold
SOURCE	Decision/Doc to origin	Provenance for answers
REQUIRES	Certificate to nodes	The minimal sufficient subgraph

Temporal handling

Edges and nodes derived from code carry commit_sha and content_hash. When a file changes, affected nodes are invalidated and re-ingested, and old edges get a valid_to timestamp rather than being deleted. This supports the Track 03 requirement to keep knowledge current and lets the evaluation replay history faithfully.

Multi-tenancy

One graph per tenant or team (for example tenant_acme_payments). Agents receive the graph name in their session context and the tool server refuses cross-graph queries. Shared, non-sensitive reference data such as language-level facts can live in a separate common graph.

6. Agent Design

Roles

Agent	Job	Reads	Writes
Triage	Classify ticket, find entry points, check playbooks and similar incidents	Ticket, Playbook, Incident	Task, AFFECTS
Localizer	Expand from entry points, rank candidates, assemble context	Code, History	Candidate set, evidence path
Fixer	Propose a patch from assembled context	Context package	Patch
Verifier	Run tests in the sandbox, decide pass or fail, request retry	Patch, Test	Result
Curator	Build certificate, record incident, promote patterns	All of the above	Incident, Certificate, Pattern, Playbook

Handoffs are represented as Task nodes with a status and owner. This makes shared state inspectable and satisfies the handoff requirement without a hidden message bus.

MCP graph tools

Tool	Returns
find_entry_points(ticket)	Candidate functions from stack trace, identifiers and text match
expand_neighbors(node, hops, edge_types)	Bounded neighborhood
callers(function), callees(function)	Call relationships
covering_tests(function)	Tests that exercise it
recent_changes(node, days)	Commits touching it
owners(node)	Likely owners and who knows it

Tool	Returns
similar_incidents(ticket)	Past incidents by pattern and text similarity
get_playbooks(pattern)	Applicable playbooks
shortest_path(a, b)	Path between error site and candidate cause
risk_rank(scope)	PageRank or centrality to flag high fan-in code
trace_decision(node)	Decisions and their sources
write_incident(...)	Persist outcome (Curator only)

Tool permissions are role-scoped: only the Curator can write, and only the Verifier can invoke the sandbox.

Retrieval and stopping

1. **Entry points** come from stack-trace frames and identifiers first, then keyword search, then embedding similarity as a fallback.
2. **Expansion** is bounded (default two hops) and type-filtered. Candidates are scored by graph distance, recent change, test coverage and past-incident links.
3. **Context assembly** adds each selected function with the signatures of its direct dependencies, within a token budget. Large outputs such as logs are trimmed to the lines relevant to the ticket, and the raw output is stored by hash so it can be re-expanded.
4. **Stopping rule.** Expansion stops when no unresolved references remain in the candidate set, or the last hop added no new high-scoring node, or the budget is reached. This is deliberately a plain rule; a learned controller is a stretch goal.

7. Differentiators in Detail

7.1 Context sufficiency certificates

After a patch passes verification:

1. Take the set C of context nodes that were loaded.
2. Remove one node (or one small group) at a time and ask the Fixer to regenerate the fix with the reduced context.

3. If the fix still passes, the node was not needed; drop it permanently from the working set.
4. Repeat greedily. The result is a minimal sufficient subgraph for that fix.
5. Store it as a Certificate with REQUIRES edges.

Cost control: cap ablation to the top-N context nodes, run it asynchronously after the fix, use a cheaper model where acceptable, and record the cost. The certificate is a measured statement (these nodes were sufficient for this fix under this model), not a proof of necessity in general.

7.2 Memory that compiles into playbooks

- **Episodic:** every Incident keeps the ticket, the path, the fix and the outcome.
- **Semantic:** durable facts such as component responsibilities and ownership, updated as evidence accumulates.
- **Procedural:** incidents are abstracted into Patterns (symptom signature, code region type, cause category, fix shape). When a pattern recurs with verified outcomes above a threshold, it is promoted to a Playbook. Playbooks are retrieved before open exploration and used as hypotheses, never as unchecked answers.
- **Safeguards:** playbooks carry success and failure counts and are demoted when they stop working or when the underlying code changes.

7.3 Learning curve evaluation

The evaluation replays a chronological stream of bugs and plots performance as memory accumulates (section 11). This is the main evidence that memory and the graph actually help.

8. Company Brain Layer (Track 03)

The code graph is extended with organizational knowledge:

- **Ingestion:** tickets, runbooks and docs, PR descriptions and review comments, and exported chat threads, linked to the code they mention.
- **Ownership:** OWNED_BY from git history and CODEOWNERS where present; KNOWS edges weighted by commits and past fixes. The agent answers who should review this and who has fixed similar issues.
- **Decision tracing:** Decision nodes extracted from PR discussions and docs, each with a SOURCE edge. Any answer that relies on a decision cites it.

- **Freshness:** stale documents are flagged when the code they describe has changed since the document's last update.

Scope control: use one real or realistic repository with its issues and PRs, and a small set of documents. Depth on one organization beats breadth across many.

9. Design Options and Trade-offs

Decision	Options	Recommendation	When to switch
Graph database	FalkorDB (required by the hackathon)	FalkorDB, one graph per tenant	Not applicable for the hackathon
Code indexing	Python ast; Tree-sitter; compiler-grade indexers (SCIP)	Python ast plus Tree-sitter fallback for one primary language	SCIP if cross-file symbol resolution errors dominate failures
Graph construction framework	Custom ingestion; GraphRAG-SDK; Graphiti; Cognee	Custom for code and history, optionally Graphiti or Cognee for the memory and knowledge layers	Use a framework if it shortens memory work without hiding the graph
Agent orchestration	Custom loop; LangChain; LlamaIndex	Thin custom orchestrator with Task nodes	A framework if tool-calling glue becomes the bottleneck
Tool interface	Direct function calls; MCP	MCP server over the graph tools	Direct calls only as a fallback
Embeddings	Hosted API; local model	Local model for reproducibility and privacy, with lexical match first	Hosted API if local quality is insufficient
LLM	Single model; strong/cheap split	Strong model for fixing, cheaper model for triage and ablation	Single model for simplicity

Decision	Options	Recommendation	When to switch
Sandbox	Local Docker; hosted microVM sandboxes	Local Docker with network disabled	Hosted sandbox if parallel verification is limited by local resources
Frontend	React + graph visualization	React with a graph view and trace timeline	Simple HTML if UI time is short
Orchestration of concurrency	Async workers; queue	Async workers with a simple queue	A real queue if load tests demand it

Guiding principle: use the graph for decisions, not for decoration. If a feature could be done equally well without the graph, it does not count toward the entry.

10. Technology Stack (Proposed)

- **Language:** Python for ingestion, agents, MCP server and evaluation; TypeScript/React for the UI.
- **Backend:** FastAPI for the API and orchestration endpoints.
- **Graph:** FalkorDB via Docker, official Python client, Cypher.
- **Agent tools:** MCP server exposing the graph tools.
- **Models:** one strong model for patch generation, one cheaper model for triage and ablation; embeddings only as a fallback retriever.
- **Testing and sandbox:** pytest in a network-disabled Docker container.
- **Evaluation:** a Python harness that writes JSONL results and generates plots.

11. Evaluation Plan

Dataset

- Primary: a chronological stream of 30-60 real bugs with failing tests from one or two open-source Python repositories, ordered by fix date. Prefer bugs fixed after the model's training cutoff, or report contamination risk explicitly.
- Secondary: 10 seeded bugs in a small demo service for the live demonstration.

Arms (paired comparisons on the same bugs)

Arm	Description
A	Whole-file or repository-level retrieval (BM25 top-k)
B	Graph localization without memory
C	Graph localization with episodic memory
D	Full system: graph, memory and promoted playbooks

Metrics

- **Localization:** whether the context contains the function actually changed by the human fix (precision and recall at function and file level).
- **Efficiency:** tokens loaded and number of tool calls per resolved ticket.
- **Outcome:** fix pass rate on the bug's tests, with regression tests.
- **Memory effect:** the above metrics plotted against the position in the stream (the learning curve).
- **Certificate quality:** size of the minimal sufficient subgraph compared with loaded context, and whether a fix regenerated from the certificate alone still passes.
- **Concurrency:** throughput and latency with N parallel ticket workers on separate tenant graphs.

Rigor

Use paired tests across arms on the same bugs, report variance across repeated runs, and publish failures. With a small sample, describe the results as a controlled study, not a benchmark claim.

12. Security and Reliability

- **Untrusted input:** ticket text, logs and documents may contain prompt injection. Treat them as data, delimit them in prompts, restrict tools by role, and never let ticket text choose tools or queries directly.

- **Code execution:** run generated patches only in a network-disabled container with resource limits.
- **Human approval:** nothing merges without a human; the agent opens a pull request.
- **Secrets and privacy:** keep API keys in environment variables, avoid sending sensitive code or tickets to external services unnecessarily, and log what leaves the system.
- **Tenant isolation:** graph-per-tenant, enforced in the tool server.
- **Auditability:** every tool call, handoff and write is logged with the Task and Session IDs.
- **Degraded modes:** if the LLM or embedding service is unavailable, the system returns the localization and evidence path without a patch.

13. Repository Structure

```
tracefix/
  README.md
  docs/                architecture, schema, evaluation protocol, blog draft
  ingest/              code parser, git history, tickets, docs, decisions
  graph/               schema, Cypher queries, migrations, tenant management
  tools/               MCP server and graph tool implementations
  agents/              triage, localizer, fixer, verifier, curator,
orchestrator
  context/             assembler, trimming, stopping rule
  certificate/         ablation engine, certificate storage
  memory/              pattern extraction, playbook promotion, demotion
  sandbox/             container runner and test harness
  eval/                datasets, replay runner, baselines, metrics, plots
  api/                 FastAPI service
  ui/                  React trace inspector and learning-curve view
  tests/               unit, integration, tenant isolation, injection tests
  configs/             model, budget and threshold settings
```

Keep ingest, tools, agents and eval separate so the evaluation can run the same agents against different baselines and so the graph tools can be tested without an LLM.

14. Work Packages for Long-Running Coding Agents

Each package has an owner agent or person, dependencies and an acceptance test, so parallel work stays verifiable.

WP	Scope	Depends on	Acceptance test
1	FalkorDB setup, schema, tenant creation	None	Create and isolate two tenant graphs; schema tests pass
2	Code and history ingestion	1	Function-level graph of the target repo with CALLS and COVERED_BY; spot-check accuracy on sampled edges
3	Ticket, doc and decision ingestion	1	Linked tickets and docs with SOURCE edges
4	MCP graph tool server	1, 2	Every tool tested against fixtures without an LLM
5	Localizer and context assembler	4	Context contains the changed function on a seed set; token counts recorded
6	Fixer, sandbox and verifier	5	End-to-end fix on seeded bugs with tests passing
7	Curator, incident memory	6	Incident written and retrievable by similar_incidents
8	Certificate engine	6	Certificate regenerates a passing fix on a sample
9	Playbook promotion	7	A recurring pattern is promoted and retrieved first
10	Evaluation harness and baselines	5-9	Replay produces all arms and the learning curve
11	Concurrency test	7	Throughput report with parallel tenants
12	Trace UI	5-9	Shows path, certificate and curve for a live run
13	Demo, video, blog	10, 12	Rehearsed run and written post

Run WP2, WP3 and WP4 in parallel after WP1, and WP8, WP9 and WP11 in parallel after WP7.

15. Demonstration Plan

1. **Ticket in (30s):** a realistic support ticket with a stack trace.
2. **Graph reasoning (60s):** the trace UI shows the traversal from error site to candidate functions, the tests covering them, the owner, and a similar past incident.
3. **Minimal context (45s):** tokens loaded compared with the whole-file baseline.
4. **Fix and verification (60s):** patch, failing test turning green, pull request opened for approval.
5. **Certificate (45s):** the minimal sufficient subgraph, and the fix regenerated from only that.
6. **Memory (45s):** a second, related ticket resolved faster using a promoted playbook.
7. **Evidence (30s):** the learning curve and a failure case reported honestly.

Record a fallback video and use a pre-ingested graph so the demo does not depend on live ingestion.

16. Risks and Mitigations

Risk	Mitigation
Model contamination on public bugs	Use post-cutoff bugs or seeded bugs and report which
Ablation cost multiplies LLM calls	Cap nodes ablated, run asynchronously, use a cheaper model
Static call graphs miss dynamic behavior	State the limitation; fall back to text and stack-trace entry points
Playbooks encode wrong lessons	Success and failure counts, demotion, always verify by tests
Scope creep across three tracks	Track 01 is the core; Tracks 02 and 03 are layers that must each justify their graph use
Prior art overlaps	Claim the specific combination and measured results, not a first

Risk	Mitigation
Small evaluation sample	Report as a controlled study with variance

17. Assumptions to Verify

- FalkorDB features (per-tenant graphs, algorithms, MCP support, client capabilities) match the hackathon page and current documentation.
- Hackathon rules: submission format, whether an existing project or coding assistants are allowed, deployment requirements, and judging criteria.
- Prior art in code-graph agents and agent memory so the novelty claim is stated accurately.
- Licensing of the chosen repositories and datasets.
- Cost and rate limits of the chosen models.

18. Submission Checklist

- Public repository with a clear README and setup using the Docker image.
- Architecture and schema documentation.
- Evaluation results with plots and the exact commands to reproduce them.
- Demo video and a rehearsed live path.
- Blog post about the project (a separate prize category).
- A short statement of limitations and honest failure analysis.